

Two-node edge HA: arbitration design

Revision 2, approved for implementation: events are delivered by a long-poll RPC instead of fixed-interval polling; lease expiry only affects a node that is holding. Builds on the discussion notes (two-node-edge-ha.pdf); the exact contract is in sbcli docs/design/two-node-arbitration.md. Code references are to ultra origin/main, the SPDK fork origin/master and sbcli origin/main as of 2026-10-09.

Why now. Today a dual-node cluster is **not** partition-safe. sbcli sets `jc_set_dual_node(true)` for every two-member cluster (`storage_node_ops.py:apply_jc_dual_node`). With the flag on, JC tolerates the loss of the only remote JM (`alg_journal.cpp: jc_ha_nmin_jms()` returns 1), so when only the link between the nodes fails, **both nodes keep accepting writes alone**. The new `remote_jm_unhealthy` events reach the control plane, but `main_distr_event_collector.py:process_event` only stores them: they carry neither `storage_id` nor `vuid`, so nothing acts on them. This design closes that gap.

1. Goals, non-goals, assumptions

- **Goal:** a two-node cluster survives the loss of either node, or of the link between them, without split brain and without EIO to clients; it heals automatically when the link returns.
- **Goal:** storage and OpenShift reach the same verdict, so VMs restart where their storage is.
- **Non-goals:** three-node behaviour (unchanged; the flag is off there); DR between sites; changing distrib's data path.
- **Assumptions:** exactly two storage nodes; the central control plane (CP) in the data center is the witness; the storage link between the nodes and the uplink to the CP are physically independent; OpenShift runs two-node with arbiter or with fencing (BMC); the edge runs the simplyblock operator in `deployment.profile: managed`.

2. Components and responsibilities

Component	Today	Adds in this design
ultra JC	<code>alg_journal.cpp</code> : JM health every 50 ms (<code>c_jc_dt_ms_check_jm_health</code>), heartbeats 100 ms (<code>bdev_jm.h</code>); health events via <code>helper_check_jm_health</code> ; dual-node tolerance	Hold trigger, lease and epoch, grant and fence RPCs, self-fence at the deadline; resume after JCERR only with a grant
ultra distrib	<code>bdev_distrib_impl.cpp</code> : event queue (<code>api_bdev_distrib_push_jm_status_event</code> , <code>distr_status_events_get</code>); <code>api_bdev_distrib_set_non_leader</code> signals the lvsore	Events carry reason, hold state and epoch; new hold/fenced/self-fenced events
SPDK lvol / blobstore	Frozen-IO queue: <code>blobstore.c:blob_execute_queued_io</code> runs held IO if leader, else EIO; <code>lvol.c:spdk_lvs_unfreeze_on_conflict</code> blocks ports and arms a 10 s rule-removal poller	Hold on JC's signal; on fence, return held IO as a path error (ANA inaccessible), not EIO; no rule removal while fenced
sbcli arbiter (new service)	-	Leases, epochs, verdicts, healing; state in FDB
Event collector (Go, new)	Python, polls every 5 s (<code>DISTR_EVENT_COLLECTOR_INTERVAL_SEC</code>)	Keeps one long-poll <code>jc_wait_events</code> open per node (CP-initiated); the node answers the moment an event is queued; forwards to the arbiter at once
Edge operator (managed profile)	Connects the edge to the CP	Applies the storage-fenced taint; reports node and remediation state
OpenShift	Node Health Check, Self Node Remediation or Fence Agents; out-of-service taint	Uses the storage verdict; BMC fencing confirms a dead node

3. The protocol

3.1 Leases and epochs

- **Lease.** Each node holds a lease from the arbiter, renewed every `lease_renew_ms=250` with TTL `lease_ttl_ms=1500` (both configurable). Renewal goes over the existing CP-to-node RPC path. A node measures expiry on its own monotonic clock from the last successful renewal; the arbiter measures from the last grant it sent plus a margin.
- **A lease matters only while a node is holding.** While the remote JM is healthy, an expired lease has no effect: missed renewals, WAN latency or a lost uplink never fence a healthy pair. Expiry counts only for a node that has lost its peer (state *holding*), where it decides self-fencing (3.2, 3.4).
- **Epoch.** One monotonic counter per cluster, stored in FDB with the verdict in one transaction. Every verdict raises it. Nodes reject any RPC whose epoch is lower than the highest they have seen, so a delayed or duplicated verdict cannot undo a newer one.

- **Safety rule.** The arbiter grants a node *solo* operation for an LVS only when the peer is fenced for that LVS: either the peer acknowledged `jc_fence` in this epoch, or the peer's lease expired more than `tvl+margin` ago, so the peer has self-fenced.

3.2 Hold

In dual-node mode, when JC marks the remote JM unhealthy it no longer just continues: it freezes the affected LVS and holds new client writes. The hold uses the existing frozen-IO queue (`blob_execute_queued_io`), because it already ends a hold in exactly the two ways a verdict needs: execute (leader) or reject (non-leader). The trigger reuses the `lvstore` signal path of `api_bdev_distrib_set_non_leader(jm_vuid, b_lvstore_signal)`. Reads from the local copy continue; journal compression and replication keep running on the local JM.

- **Hold deadline** `hold_ms=2500`: must stay below the hosts' NVMe keep-alive (`KATO constants.KATO=5000` ms; the port-fence incident showed a 12 s fence past a 4 s client KATO ends in EIO).
- **Grant** before the deadline: unfreeze, execute held IO, continue solo with the epoch.
- **Fence** before the deadline: set non-leader, block the LVS ports, return held IO as a path error so hosts retry on the peer's path.
- **Deadline without verdict**: self-fence, unless the preferred-node rule applies (3.4).

3.3 Verdicts

Verdict	Sent to	Effect on the node
<code>grant_solo{epoch, lvs[], ttl}</code>	survivor or partition winner	Release hold, lead alone, renew with the epoch
<code>fence{epoch, lvs[]}</code>	partition loser	Non-leader, ports blocked, held IO -> path error, JC stops appending for those LVS
<code>abort{epoch}</code>	a node that holds or leads against the epoch	Existing <code>spdk_abort_node()</code> path
<code>unfence{epoch}</code>	healed node, after the resync gate	Rejoin as secondary through the restart flow (section 6)

3.4 Both links lost: the preferred node

If neither node reaches the CP nor the other, nobody can arbitrate. One node per cluster is marked **preferred** at setup. At the hold deadline the preferred node continues solo, and the other self-fences when its lease expires. This is safe only if the arbiter never grants the non-preferred node solo operation merely because the preferred node went silent: it needs **positive** evidence that the preferred node is down, namely a successful BMC fence (OpenShift Fence Agents Remediation) or an operator-set out-of-service taint. Consequence: failover away from the preferred node waits for fencing; failover away from the other node needs only its lease to expire.

3.5 Hazards to displace

- `lvb.c:spdk_lvs_unfreeze_on_conflict` arms `spdk_lvs_remove_rules_poller` (10 s) that lifts the `hublvol` reject rule. In dual-node mode it must not lift while the node is fenced in the current epoch.
- JC resumes from error mode when `n_jms >= jc_ha_nmin_jms_resume()`, which is 1 in dual-node mode (`alg_journal.cpp ~8564`). A fenced node must not resume on its own: resume only with `grant_solo` or `unfence`.
- The earlier-noted 10 s automatic re-promotion of a non-leader was not found as such on SPDK master; the SPDK team should confirm no other timer re-promotes a fenced LVS.
- The network-outage abort (`alg_journal.cpp ~2259`, 0 JMs left in dual-node mode) stays as is.

4. Interfaces

4.1 Node RPCs (ultra JC; names proposed)

RPC	Arguments	Returns / errors
jc_set_dual_node (extended)	enable, preferred, hold_ms, lease_ttl_ms	ok; existing callers keep working (new fields optional)
jc_lease_renew	epoch, ttl_ms	{epoch_seen, state}; ESTALE if epoch < seen
jc_grant_solo	epoch, lvs[] (jm_vuid), ttl_ms	ok; ESTALE; EBUSY if fenced in this epoch
jc_fence	epoch, lvs[]	ok once ports are blocked and held IO is returned; ESTALE
jc_unfence	epoch, lvs[]	ok; EAGAIN until JC resync of the remote JM is complete
jc_ha_status	-	{state: normal holding solo fenced, epoch, since_ms, held_ios, lease_left_ms, remote_jm:{name, health, reason}}
jc_wait_events	instance, after_seq, timeout_ms, max	{instance, seq, events[], overflow}; answers at once if events after after_seq exist, else when one arrives or at timeout_ms

4.2 Event delivery: long-poll

- **Direction.** Still CP-initiated: the collector calls `jc_wait_events` on each node over the existing CP-to-node RPC path. The node does not answer at once when nothing is queued; it holds the request and answers the moment an event is pushed, or with an empty list at `timeout_ms` (default 20 s). Event-to-arbiter latency becomes one network trip instead of up to a polling interval, and an idle cluster costs one request per node per 20 s.
- **Sequence and restarts.** Every event gets a seq that increases by one per node process; the answer also carries an instance id generated at SPDK start. The collector passes the last seq it processed as `after_seq` (acknowledgement). A changed instance means the node restarted: the collector resyncs from `jc_ha_status` and starts again from `after_seq=0`.
- **Delivery.** At least once. The node keeps events after `after_seq` in a bounded ring (default 1024); if it overflows, the answer sets `overflow: true` and the collector resyncs from `jc_ha_status`. The arbiter deduplicates by (instance, seq) and ignores stale epochs.
- **Content.** `remote_jm_unhealthy` / `remote_jm_healthy` (with reason, ha_state, epoch), hold started, hold deadline reached, self-fenced, lease expired while holding, grant and fence applied, unfence completed.
- **Concurrency.** One waiter per node: a second call completes the first at once with an empty list (superseded) and takes its place, so a collector that reconnects never leaves a stale waiter behind.
- **SPDK side.** The RPC handler keeps the `spdk_jsonrpc_request` instead of answering (deferred response) and arms a timer poller on the RPC thread. Pushing an event from any thread sends a message to the RPC thread, which completes the waiting request; timeout and cancellation also run there.
- **Collector side (Go).** One goroutine per node keeps a long-poll open; on error it reconnects with backoff (100 ms doubling to 5 s). Nodes without the RPC (capability probe: `rpc_get_methods` lists `jc_wait_events`) are read with `distr_status_events_get` every 1 s, as a fallback.

4.3 Events

Today a JC event serializes as below (`bdev_distrib_impl.cpp: _s_rpc_bdev_distrib_status_events_get`; the status string comes from `dst_to_string`, so it is `remote_jm_unhealthy`, not the `jm_unhealthy` named in a header comment). Proposed additions in bold:

```
{"timestamp": "...Z", "event_type": "device_status", "jm_vuid": 3,
 "status": "remote_jm_unhealthy", "index": 0, "count": 1,
 "reason": "unavailable|blocked|out_of_sync", "ha_state": "holding", "epoch": 7}
new statuses: "ha_fenced", "ha_self_fenced", "ha_solo", "ha_hold_expired"
```

4.4 Control plane

- **FDB record** ClusterArbitration: cluster id, epoch, per-LVS holder and state, lease expiry per node, preferred node, verdict history.
- **API GET** `/clusters/{id}/arbitration` (state for console and operator); **POST** `.../arbitration/preferred`; **POST** `.../arbitration/override` for an operator decision (audited).
- **Events** in the cluster log for every verdict, with the signals it was based on.

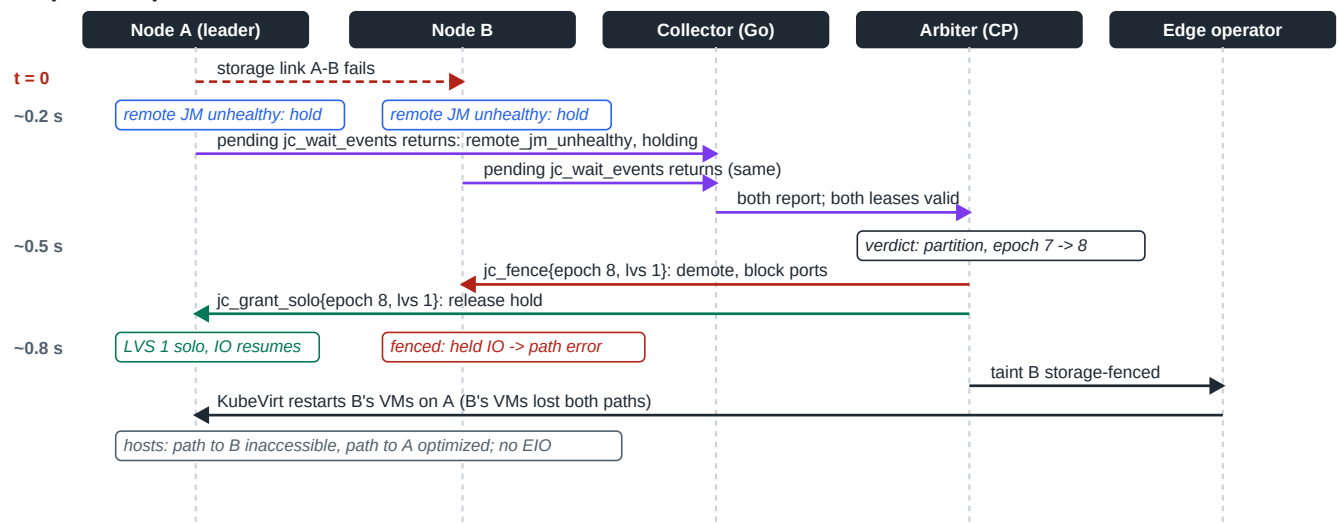
4.5 State machines

Node (per LVS)	Transition
normal -> holding	remote JM unhealthy in dual-node mode
holding -> solo	grant_solo (epoch current) before the deadline; or deadline and preferred node
holding -> fenced	fence; or deadline without a verdict (non-preferred); or lease expired while holding
solo -> normal	unfence of the peer completed, remote JM healthy again
fenced -> normal	unfence after the resync gate (section 6)
Arbiter (per cluster)	Transition
steady -> deciding	an unhealthy, hold or lease-expiry signal arrives
deciding -> partitioned	both nodes report each other and both leases are valid: fence the loser, grant the winner
deciding -> degraded	one node silent and its lease expired (or positively fenced): grant the survivor
partitioned/degraded -> healing	both report remote_jm_healthy and stay healthy for stable_for=30s
healing -> steady	resync gate passed, unfence done, optional fallback done

4.6 Timing budget

Step	Target	Basis
Detect remote JM loss	0.1 - 0.3 s	JM heartbeat 100 ms, health check 50 ms
Event to arbiter	~ 1 network trip	pending jc_wait_events answers at once (today: up to 5 s polling)
Decision + FDB write	<= 0.1 s	one transaction
Verdict RPCs over WAN	0.1 - 0.3 s	fence first, then grant
Wait for peer lease (node-death case only)	<= 1.5 s	lease_ttl_ms
Total client-visible pause	< 2.5 s	hold_ms deadline, below the 4-5 s host KATO

Sequence: partition of the link between the nodes



5. Decision matrix

A reports B	B reports A	Leases A / B	Verdict
unhealthy	silent	valid / expired	B down or isolated. Grant A solo after B's lease expiry (B self-fenced). If B is the preferred node: wait for positive fencing of B.
unhealthy	unhealthy	valid / valid	Partition of the storage link. Per LVS the current leader wins; loser fenced; the loser node gets the storage-fenced taint so its VMs restart on the winner.
healthy	healthy	expired / expired	Only the uplink is down. No verdict; nodes keep working. Expired leases have no effect while the remote JM is healthy: missed renewals never fence a healthy pair.
unhealthy	unhealthy	expired / expired	Both links down. Preferred node continues at the hold deadline; the other self-fences.
flapping	flapping	any	Hysteresis: a JM must stay healthy stable_for before healing; events are deduplicated by epoch and sequence.

Late or duplicate events are ignored by epoch. If the arbiter restarts or fails over mid-decision, it reloads `ClusterArbitration` from FDB, raises the epoch and re-sends the last verdict; nodes in hold keep holding until the deadline, which bounds the effect.

6. Healing and re-admission

1. **Detect.** Both nodes report `remote_jm_healthy`; the arbiter enters *healing* after `stable_for`.
2. **Resync gate.** JC replicates the missing journal to the returning JM; distrib rebuilds the writes the fenced node missed. `jc_unfence` returns `EAGAIN` until both are complete.
3. **Re-admit.** Reuse the restart flow on the fenced node: `storage_node_ops.py:recreate_lvstore_on_non_leader / recreate_lvstore_on_sec`, `hublvol` reconnect, non-optimized ANA path restored, then unfence.
4. **Failback (optional).** Move leadership back to balance load as a planned handover, with the quiesce of the acting leader from `sbcli #1439` (`_presumed_acting_leader: jc_disable_replication`, `bdev_distrib_check_inflight_io`, set non-leader).
5. **OpenShift.** Remove the storage-fenced taint; VMs stay where they are unless live-migrated.

7. OpenShift alignment

- **Storage decides first, in seconds;** OpenShift node remediation takes tens of seconds. Storage verdicts never wait for OpenShift, except the positive-fencing requirement for the preferred node (3.4).
- **Node death:** the survivor gets solo storage within the budget. Node Health Check marks the dead node; Self Node Remediation or Fence Agents Remediation fences it; the out-of-service taint lets volumes detach; KubeVirt restarts the VMs on the survivor. A BMC fence also counts as positive evidence for the arbiter.
- **Storage-link partition with both nodes alive:** the loser's VMs lose both paths (local fenced, remote unreachable). The arbiter asks the edge operator to taint the loser `storage.simplyblock.io/fenced:NoExecute`; KubeVirt restarts those VMs on the winner. The loser is alive, so volumes detach cleanly.
- **Cluster network placement:** keep the OpenShift cluster network on the uplink side, not only on the storage link, so a storage partition does not also split etcd.

8. Failure modes

Failure	Expected outcome
Uplink to the CP down	Storage runs normally; no verdicts possible; alerts in the CP. Missed lease renewals do not fence a healthy pair.
Storage link down	Partition verdict within ~1 s; loser fenced and tainted; its VMs restart on the winner.
Both links down	Preferred node continues after the hold deadline; the other self-fences.
Node crash	Survivor solo after the peer's lease expiry (non-preferred) or after positive fencing (preferred).
Node hang (alive, not responding)	Like a crash for storage: no renewals, lease expires, the node self-fences if it wakes; JC thread-hang abort stays.
CP down	No new verdicts; nodes in hold use the deadline rules; steady clusters unaffected.
CP failover mid-arbitration	New arbiter reloads FDB state, raises the epoch, re-sends the verdict; stale RPCs rejected by epoch.
WAN latency spikes	Harmless while the remote JM is healthy. Only a spike beyond <code>lease_ttl_ms</code> that coincides with a hold (peer lost) can fence a node that is actually fine: safe, not available. Tune <code>lease_ttl_ms</code> per site.

9. Test plan and rollout

- **Unit:** arbiter decision table, epochs, lease arithmetic with injected clocks, FDB reload; ultra hold/grant/fence state machine.
- **Ultra-level:** two SPDK instances with `tc/iptables` cutting the JM link; held IO completes on grant and returns path errors on fence; no EIO.
- **Bed:** two-node edge cluster with the CP in another VPC; cases: storage link cut, uplink cut, both, node power-off, node freeze (SIGSTOP), CP restart mid-hold, latency injection; fio with checksums on both nodes' VMs; assert no split-brain writes and no EIO.
- **OpenShift:** two-node with arbiter and two-node with fencing; VM restart after taint and after BMC fencing.
- **Rollout:** feature flag `cluster.two_node_arbitration` (off: today's behaviour); ultra and sbcli versions gated by a capability probe (`rpc_get_methods` lists `jc_ha_status` and `jc_wait_events`).

10. Work packages and open questions

Repo	Work	Size
ultra	Hold trigger, lease/epoch, grant/fence/unfence, ha_status, jc_wait_events (deferred response, ring, seq/instance), event fields, resume gating after JCERR	L
spdk fork	Hold on JC signal via the frozen-IO queue; path error instead of EIO on fence; gate the 10 s rule-removal poller	M
sbcli	Arbiter service, FDB model, API, verdict RPCs, healing flow, flag; route JC events to the arbiter	L
sbcli	Go event collector (one long-poll per node, backoff, resync on instance change, fallback polling for old nodes), packaged in the CP image	M
operator	Storage-fenced taint, remediation status to the CP, preferred-node setting in the CR	S
test	Bed scenarios, OpenShift two-node variants, latency injection	M

Open questions

- **For Alireza:** can JC trigger the lvstore freeze directly from helper_check_jm_health, or should distrib do it on the event? Is holding user writes while JC keeps compressing and replicating on the local JM safe for the journal?
- **For Alireza:** confirm the event string (remote_jm_unhealthy per dst_to_string) and fix the stale header comment; add reason and epoch fields.
- **For the SPDK team:** returning held IO as an ANA/path error instead of EIO on fence; confirm no timer re-promotes a fenced LVS.
- **Team:** defaults for hold_ms, lease_ttl_ms and stable_for over typical edge WAN links; how the preferred node is chosen (static per cluster, or the node with more leaders).
- **Team:** where the arbiter runs (inside the CP task runner, or its own deployment for latency and isolation).